Cobalt Instrument Studio

Engineering Report

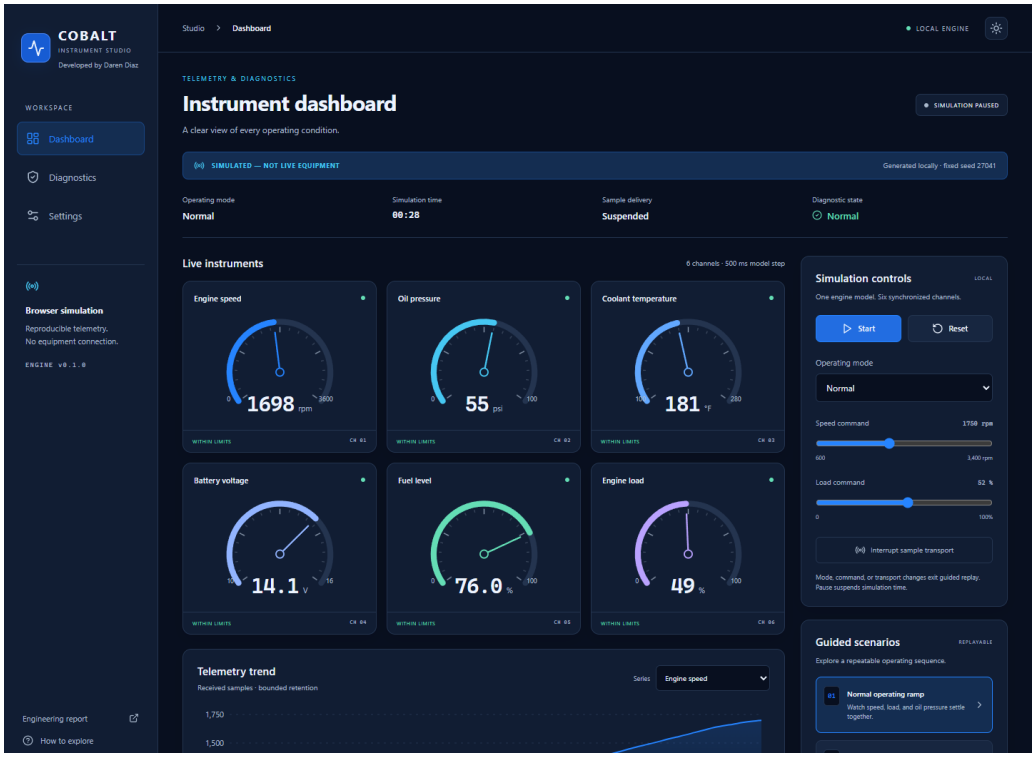
Developed by Daren Diaz

Version 0.1.0 | Fall 2026

Purpose and implemented behavior

Cobalt Instrument Studio demonstrates how a single, coherent telemetry model can support an industrial display, reproducible fault scenarios, and inspectable diagnostic behavior. The implementation runs entirely in the visitor's browser as a static React and TypeScript application. It provides a working exploration environment for instrumentation and software design.

The local implementation includes six synchronized instruments, four operating modes, three replayable scenarios, validated thresholds, acknowledgment and recovery rules, bounded history, local settings, and CSV export. Validation recorded 32 unit tests and 7 browser tests passing. Public hosting, hardware integration, and production validation remain outside the implemented capability.



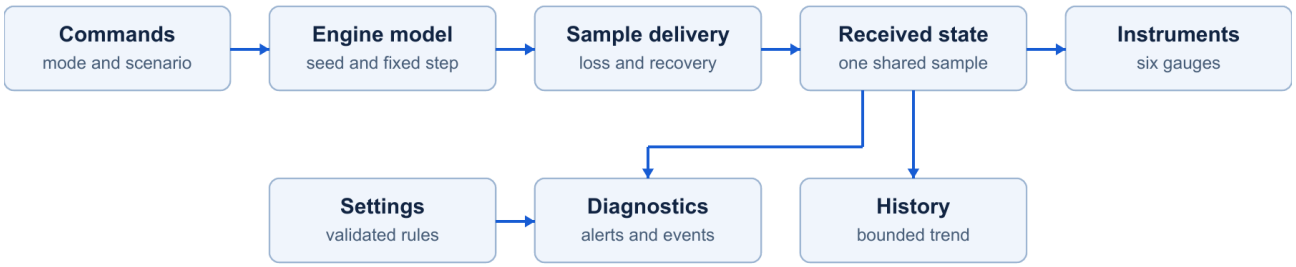
Actual desktop browser capture of the simulator during a paused normal operating ramp. The image is cropped for readability.

Simulation scope

SIMULATED - NOT LIVE EQUIPMENT. Every instrument value is generated locally. The interface is a display and simulator, with no backend, device connection, equipment command path, user authentication, or safety control. The telemetry relationships are engineering assumptions for this demonstration; they are not calibrated equipment measurements.

Architecture and state ownership

One typed reducer owns the session. Modes and guided scenarios supply commands to a seeded engine model. Simulated transport determines whether a new sample is delivered. Gauges, diagnostic rules, the chart, and export consume the same last received telemetry, so a communication interruption freezes displayed values without stopping the internal model.



One model and one delivery path feed the display and diagnostic system. Settings change presentation and validated rules.

Component	Current capability	Engineering value	Generic application	Production gap
Engine model	Seeded six-channel state and four modes	Reproducible coupled behavior	Generator operating ramp training	Calibrated equipment model and real sensors
Sample transport	Deterministic outage and recovery	Separates generated data from received data	Gateway timeout workflow exploration	Sequence numbers, real timestamps, network integrity
Diagnostic rules	Thresholds, hysteresis, debounce, acknowledgment	Explains onset, escalation, and recovery	Fault triage workflow prototyping	Validated alarm policy and equipment hazard analysis
Display and history	SVG gauges, one selectable trend, bounded buffers	Makes state and retention inspectable	Pump or motor condition-display evaluation	Formal accessibility, device, and performance validation
Settings and export	Validated preferences and local simulated CSV	Enables repeatable configuration and review	Review of operating sequences	Authenticated configuration, durable audit and retention
Release preparation	CI and tag/version verification files	Reviewable checks and version traceability	Repeatable software build review	Verified remote runs and deployment safeguards

The engine and diagnostic rules are pure modules under src/engine. Components render reducer state. LocalStorage stores only validated settings. Charts and events stay in bounded memory, and exports download through browser Blob URLs. No server API or network data source is required during site operation.

Telemetry assumptions and invariants

The model advances by a fixed 0.5 simulated seconds per delivered timer step. This preserves exact seeded replay even if wall-clock scheduling varies. Pause stops simulation time and timeout age. The 500 ms interval is a configured target, not a real-time scheduling guarantee; background tabs may be throttled.

Channel	Assumed relationship	Guarded range
RPM	Approaches the speed command with a 0.14 per-step response plus bounded seeded disturbance.	600 to 3600 rpm
Engine load	Approaches the load command with a 0.12 per-step response.	0 to 100 percent
Oil pressure	Pressure target rises with RPM and decreases with load and coolant above 210 degrees F. Fault mode targets low pressure.	0 to 100 psi
Coolant	First-order temperature inertia toward an RPM/load target. Cooling failure raises the target to 272 degrees F.	100 to 280 degrees F
Battery	Charging target is 13.2 V at idle or 14.1 V at higher speed; Fault mode targets 11.4 V.	10 to 16 V
Fuel	Declines monotonically at 0.0008 plus load times 0.000025 percentage points per simulated second.	0 to 100 percent

Time and units

Canonical model units are rpm, psi, degrees F, V, and percent. Metric display converts pressure to kPa and temperature to degrees C without changing model values or diagnostic thresholds. The settings editor explicitly retains canonical threshold units. Scenario fault kinetics are accelerated for inspection and do not establish a physical time constant.

Bounded retention

Sample history retains at most 240 received samples, representing up to roughly 120 simulated seconds at full delivery. The timestamp span can be longer across an outage because missing samples are not appended. Events retain at most 100 records. The trend leaves a gap across undelivered samples rather than interpolating a false trace.

Model limitations

There is no engine shutdown model, sensor calibration, noise characterization from hardware, environmental model, fuel mass balance, or validated charging circuit. Seeded disturbance provides repeatability, not statistical realism. Fault values and display ranges support an explanatory demonstration and must not be reused as equipment operating limits.

Reproducible scenarios and diagnostic behavior

Normal operating ramp

Expected behavior: begin at idle, raise speed and load smoothly, let oil pressure follow, and retain slow thermal response. Observed with seed 27041 at 35 simulated seconds: RPM 1749.8, load 51.8 percent, oil 57.1 psi, and coolant 182.7 degrees F. No threshold transition occurred under the default rules. Replay resets the model, history, and diagnostics to the same sequence while retaining preferences.

Coolant overheat

Expected behavior: cooling degradation begins at 6 s and progressively crosses warning and critical thresholds. Observed transitions were normal to warning at 14.0 s with 223.5 degrees F, then warning to critical at 18.5 s with 236.8 degrees F. The default boundaries are 220 and 235 degrees F with a 4 degree F recovery margin and two received samples of debounce. At 35 s the model reached 261.1 degrees F. Browser checks acknowledged the warning, verified renewed acknowledgment after escalation, and reset the session to normal.

Communication loss

Expected behavior: drop delivered samples from 5 s through 16.5 s while the central model continues. The final sample before the gap was received at 4.5 s. Observed timeout onset was 6.5 s, exactly 2 simulated seconds after that sample. Gauges retained stale received values and history stopped growing. At 17 s a fresh sample arrived and the communication condition resolved automatically. The recovered trend preserved the delivery gap.

Rule state and acknowledgment

Each channel maintains one normal, warning, or critical state. Hysteresis defines a separate recovery boundary; configurable debounce requires consecutive received samples. Stale duplicates do not count toward telemetry debounce. Communication timeout is immediate when its age boundary is reached. Transition events explain the triggering value and threshold. Acknowledgment records review, leaves the continuing condition active, and resets on escalation or recovery. Sustained alerts are deduplicated to prevent event floods.

All timestamps above are simulation time. Automated guided browser checks used a controlled clock. These observations establish application behavior for a defined seed and settings, not real machinery response or field communications reliability.

Validation and measured application behavior

Local validation completed 32 unit checks and 7 browser checks with zero failures in the final baseline. TypeScript and production compilation were checked separately. Tests exercise behavior through both the pure engine and the actual rendered application.

Requirement	Method and actual result	Status
Six coherent instruments	Bounded model runs in all modes plus six rendered browser gauges.	Pass
Seeded replay and mode controls	Exact replay equality; start, pause, reset, and live mode change.	Pass
Three guided scenarios	Normal, warning/critical overheat, stale data, and automatic recovery.	Pass
Reliable diagnostics	Exact boundaries, recovery margin, debounce, deduplication, and acknowledgment.	Pass
Bounded memory structures	240-sample history after 1500 ticks; 100-event cap after 300 events.	Pass
Settings and persistence	Malformed/blocked storage fallback; actual browser reload with saved preferences.	Pass
Desktop and mobile	1440 px desktop and 390 px mobile; no horizontal overflow in three views.	Pass
CSV evidence	Actual download includes explicit simulated-data label.	Pass
Root and project builds	Both local static production paths passed browser checks and matching report downloads.	Pass
DOCX visual layout	Seven Word pages rendered and inspected; blank page corrected.	Pass
Privacy	Restricted identity text scan, blank report author fields, and real screenshot review.	Pass
Remote CI and hosting	Prepared workflow and hosting files; no remote run or publication.	Not run

Measured observations

Production JavaScript totaled 625,760 bytes (188,921 bytes gzip); CSS totaled 23,748 bytes. In one desktop smoke check, the simulation clock reached 3 s after 3626 ms of wall time including interaction overhead, with no page errors. This measures one workflow, not interval accuracy.

Browser tests took 34.7 s; unit test bodies took 2.47 s. Heap, frame rate, browser soak, hardware timing, and public availability were not measured. Buffer caps do not prove a total heap bound.

Current demonstration and production requirements

The project makes instrumentation behavior reviewable without access to equipment. A real industrial deployment would require separate integration, safety, security, and reliability engineering. The comparison below identifies future work rather than claiming production readiness.

Area	Implemented demonstration	Hypothetical production deployment
Connectivity	Locally generated values and simulated delivery interruption.	Validated gateway/device adapter, framing, unit schema, real timestamps, sequence checks, and link testing. CAN/J1939 is only a hypothetical interface.
Safety	Display and simulator controls with a persistent simulation notice.	Equipment hazard analysis, separated safety/control paths, safe failure behavior, independent review, and applicable verification.
Security	No equipment, backend, authentication, or embedded credentials.	Device and user authentication, authorized changes, secret lifecycle, integrity checks, transport protections, and isolation.
Diagnostics	Configurable thresholds, hysteresis, debounce, and acknowledgment.	Equipment-specific alert policy, calibrated boundaries, nuisance-alarm analysis, routing, and validated operator response.
Logging	Bounded session memory and locally downloaded simulated CSV.	Durable timestamped storage, retention, access control, integrity, accountable audit, and operational monitoring.
Testing	Deterministic model tests and local Edge browser paths.	Hardware in the loop, calibrated sensor tests, network impairment, cross-browser/device checks, soak tests, and recovery exercises.
Reliability	Fixed simulation step and prepared static hosting builds.	Defined service objectives, power/network failure behavior, deployment safeguards, monitoring, backup, and rollback.

Design tradeoffs

A static frontend keeps operation inspectable and avoids server, credential, and device dependencies. One central reducer makes relationships and replay behavior explicit. Fixed steps favor deterministic demonstration over real-time fidelity. Bounded buffers keep retention deliberate, while memory-only telemetry avoids implying durable records. A single selectable chart and three simple views keep the scope focused.

The application can help evaluate a display or diagnostic workflow before implementing equipment connectivity. It cannot command a physical machine, prove a hardware fault, certify an alarm policy, or replace an operational monitoring system.

Generic applications and future engineering work

Operator workflow examples

Generator instrumentation training: an operator can run the normal ramp, compare speed, pressure, and load, then follow an overheat escalation without connecting equipment. Production use would require validated sensor mapping, equipment-specific thresholds, and a separate control/safety design.

Fleet diagnostic review: a reviewer can inspect stale readings, acknowledge a communication alert, and trace recovery events. A real fleet system would need authenticated multi-device identity, reliable timestamps, durable records, and permissions.

Pump or motor condition-display prototyping: a designer can evaluate layout, readable units, fault presentation, and replay behavior. Real application would require a different validated telemetry model, physical sensors, hardware tests, and failure analysis.

Prioritized potential improvements

P0: real gateway/fieldbus adapter; validated telemetry schema; hardware in the loop; connectivity security hardening; expanded end to end and cross-browser tests. These establish prerequisites for receiving real data safely and interpreting it correctly.

P1: advanced fault injection; historical storage; alert routing; role based access; audit trails; multi-device support; device configuration. Each is future work beyond the implemented browser session and three basic replays.

P2: performance profiling; formal accessibility audit; automated release safeguards. The separate roadmap gives engineering benefit and rough Small/Medium/Large complexity for all 15 proposed improvements. No delivery commitment is implied.

Version and publication state

Version 0.1.0 is prepared locally. CI, GitHub Pages deployment, issue forms, and a tag-triggered release workflow are present but have not run remotely. Netlify production is the intended primary shareable endpoint and Pages is the independent backup. Public repository creation, commits/pushes, site deployment, and the v0.1.0 tag/release require a separate authorized publication step after inspection.

Both hosted builds will serve static assets and downloadable reports, without requiring the development computer to remain powered on. Provider permissions, quotas, plan limits, and future availability must be checked during publication; no uptime guarantee or verified public endpoint exists yet.